

PRD: The "Ignite" FIRE Engine Integration

1. Executive Summary

The "Ignite" project aims to integrate a Financial Independence, Retire Early (FIRE) simulation engine into the existing financial app. By leveraging real-time balance data from Monarch Money, the app will provide users with a "Retirement Probability Score," an estimated "FIRE Date," and a dynamic "Safe Withdrawal Rate" based on market volatility.

2. Problem Statement

Users have their data (balances, transactions) but lack the "so what?" They don't know if their current trajectory leads to retirement at 45 or 65. Most online calculators require manual entry of balances, which quickly becomes outdated.

3. User Stories

- **As a user**, I want my current net worth to automatically sync so my FIRE projections are always up-to-date.
- **As a user**, I want to see a range of outcomes (Monte Carlo) rather than a single "best-case" number.
- **As a user**, I want to know how much I can safely spend today without jeopardizing my future retirement.

4. Functional Requirements

- **Data Ingestion:** Fetch total liquid assets, brokerage balances, and current monthly spend from the Monarch Money API.
 - **Simulation Engine:** Run 10,000+ Monte Carlo iterations using historical market returns and inflation.
 - **SWR Calculator:** Determine the Safe Withdrawal Rate (SWR) using the Trinity Study logic, adjustable for "Fat FIRE" or "Lean FIRE" risk tolerances.
 - **Visualization:** Display a "cone of uncertainty" showing portfolio growth over time.
-
-

5. Recommended Open Source Libraries (Python)

To build this without reinventing the wheel, I recommend the following stack:

A. Data Connection: [monarchmoney](#)

This is the unofficial but highly effective Python wrapper for the Monarch Money API.

- **Purpose:** To programmatically pull your `get_accounts()` and `get_cashflow_summary()` data.
- **Why:** It handles the MFA and session management required to get your "ground truth" balances.

B. The Simulation Engine: `monteplan`

Released/updated in early 2026, this is currently the most robust specialized library for this task.

- **Purpose:** Vectorized Monte Carlo simulations using `numpy`.
- **Features:** It supports multivariate normal distributions and "Historical Block Bootstrap" (using actual sequences of historical market data rather than just random averages), which is crucial for accounting for "Sequence of Returns Risk."

C. Optimization & Tax Logic: `Owl` (Owl Planner)

If your app needs to be "smart" about taxes (e.g., drawing from a 401k vs. a Roth IRA).

- **Purpose:** It uses linear programming optimization to determine the most tax-efficient withdrawal strategy.
- **Why:** Retirement isn't just about the "number"—it's about the net amount after the government takes its share.

D. Core Math: `numpy` & `pandas`

- **Purpose:** Standard data manipulation and high-speed financial calculations.

6. Technical Implementation Sketch

Python

```
import monarchmoney as mm
from monteplan.engine import MonteCarloEngine
```

```
# 1. Fetch live balances from Monarch
```

```
client = mm.MonarchMoney()
```

```
# ... authentication ...
```

```
accounts = await client.get_accounts()
```

```
current_nw = sum(acc['balance'] for acc in accounts if acc['type'] == 'investment')
```

```
# 2. Configure Simulation
```

```
engine = MonteCarloEngine(
```

```
    initial_portfolio=current_nw,
```

```
    annual_contribution=50000,  
    years_to_sim=40,  
    iterations=10000  
)
```

3. Run and Analyze

```
results = engine.run_historical_bootstrap()  
print(f"95% Success Safe Withdrawal Rate: {results.swr_95}%")
```

